GOOGLE · 2026

One Environment, N Machines

Deterministic builds for humans *and* agents

Steve Morin VP Eng, Flox @smorin

THE PROBLEM

Where is your software being built?

⌨

A laptop

Set up by a person. Slowly. Once. Then drifted for two years.

`set up by: a human`

⟳

A fleet of CI runners

Set up by a YAML file and whatever the base image had that morning.

`set up by: a script`

▣

A local sandbox

Set up by a container image someone built, somewhere, from a Dockerfile nobody reads.

`set up by: an image`

✦

A remote sandbox

Set up by the agent that is about to run your tests in it. It is guessing.

`set up by: the agent`

Four machines, four setup stories, one codebase.

1:00

Nobody sets these machines up by hand anymore.

And when they do, every one of them does it a little differently.

SETUP, TODAY

Four machines, four install stories

```
# macOS
brew install python@3.14 redis jq
npm install -g @anthropic-ai/claude-code

# CI (ubuntu-latest)
sudo apt-get install -y python3 redis-server jq   # python 3.12, redis 7.0
curl -fsSL https://example.invalid/install.sh | sh

# Dockerfile
FROM python:3.13-slim                             # a third python
RUN apt-get update && apt-get install -y redis jq

# The agent
# ...reads all of the above, picks one, and hopes.
```

Three Pythons, two Redises, and a curl | sh. Same repo.

THAT MESS IS TWO PROBLEMS

Build it the same. Run it the same.

1. Same build, every machine

Laptop, CI runner, local sandbox, agent sandbox. One definition of the toolchain, reproduced exactly, so a green build means the same thing everywhere.

2. Same run, wherever it lands next

Another runner, a container, production, or six months from now. The artifact carries its own closure, so it does not depend on what the host happened to have.

The model's reasoning is the only part that should vary.

THE FIX

Declare it. Don't script it.

`manifest.toml`
you write this

→

`manifest.lock`
exact versions, every system

→

`/nix/store/7c18d0…`
hash-pinned closure

`flox activate`
dev shell, services

`flox build`
hermetic artifact

`flox containerize`
OCI image, no Dockerfile

`flox publish`
share the package

✔ Declarative ✔ Deterministic ✔ Verifiable

The engine underneath is Nix. Next: how it works, and where Flox sits on it.

UNDER THE HOOD

What Nix actually does.

Every package lives at a path that is a hash of its inputs.

`/nix/store/0kcws5blwbdyxqxjd1w8hjipz61xny3k-python3-3.14.7`

hash of the source, every dependency, the build script, the flags, and the target system **name** for humans only

- Nothing goes in `/usr/lib`. Each build gets its own directory, written once, never edited.
- Dependencies are absolute store paths fixed at build time. No search path, so nothing to "find".
- Change any input and you get a new hash beside the old one. Two Pythons never fight.

The real path of the Python in this repo, from manifest.lock.

laptop · ~/deterministic-builds-google aarch64-darwin

```
$ flox activate -- sh -c 'realpath "$(which python3)"'
/nix/store/0kcws5blwbdyxqxjd1w8hjipz61xny3k-python3-3.14.7/bin/python3.14
$ nix-store -q --references /nix/store/0kcws5blwbdyxqxjd1w8hjipz61xny3k-python3-3.14.7 | head -6
/nix/store/0fz1gcmyam04pahr91fzjph2c5mi6rv0-bzip2-1.0.8
/nix/store/phb97pafnka8z1qpj12is214rshpql65-zlib-1.3.2
/nix/store/0k4r9hnfbnyngn5f3j31siddfs0iglwl-sqlite-3.53.3
/nix/store/2xyjpks2z1i91klny38h789j483fpz4x-libffi-40
/nix/store/8kn5h4pkns2wf662nvifg08vvz41c0dl-zstd-1.5.7
/nix/store/9496m45iskps77ngzq2i16a7snig4dx9-xz-5.8.4
$ nix-store -q --requisites /nix/store/0kcws5blwbdyxqxjd1w8hjipz61xny3k-python3-3.14.7 | wc -l
      51
$ nix-store -q --deriver /nix/store/0kcws5blwbdyxqxjd1w8hjipz61xny3k-python3-3.14.7
/nix/store/p745p2wn875yp2k6nf6rvpbmc6w6qcw9-python3-3.14.7.drv
$ nix derivation show /nix/store/p745p2wn875yp2k6nf6rvpbmc6w6qcw9-python3-3.14.7.drv | jq '.[] | {system, builder,
inputDrvs: (.inputDrvs|length), inputSrcs: (.inputSrcs|length), out: .outputs.out.path}'
{
  "system": "aarch64-darwin",
  "builder": "/nix/store/f0b9dbw2gl596cm03pvdsss72381qhlp-bash-5.3p15/bin/bash",
  "inputDrvs": 21,
  "inputSrcs": 6,
  "out": "/nix/store/0kcws5blwbdyxqxjd1w8hjipz61xny3k-python3-3.14.7"
}
```

What Nix gives you. What it asks of you.

Gives

A store where every path is a hash of its inputs

nixpkgs: 100,000+ packages, one repo, one commit to pin

Binary caches: build once, fetch the same bytes everywhere

Sandboxed builds: no network, no `$HOME`, no `/usr`

Asks

Learn the Nix language: lazy, functional, its own thing

Pick a pinning story: channels, flakes, or pinned tarballs

Write a derivation for anything not already in nixpkgs

Think per system: the hash includes the platform

Flox is one answer to the right column. devenv, devbox, and plain flakes are others.

NIX, PART 3 · WHERE FLOX SITS

Flox on top of Nix.

`manifest.toml` you
names, version constraints, vars, services, a build recipe, target systems

`flox` CLI · GPL-2.0
resolves names to a nixpkgs commit, writes manifest.lock with a store path per system, composes the environment, drives the sandboxed build and the OCI image

`nix` LGPL-2.1
evaluates derivations, fetches from a binary cache or builds, owns /nix/store

`nixpkgs` MIT
100,000+ package recipes in one git repository, pinned by commit

Flox adds

- TOML instead of the Nix language
- Version constraints, resolved against an index of every nixpkgs revision
- One lock for every system
- Vars, services, build, containerize in the same file
- Push and pull whole environments

Flox does not change

- The store
- The hashes
- The packages

Every store path in manifest.lock is one nix build away, with or without Flox.

```
laptop · ~/deterministic-builds-google                    aarch64-darwin

$ jq -r '.packages[] | select(.install_id=="python3" and .system=="aarch64-darwin") |
.locked_url, .attr_path, .outputs.out' .flox/env/manifest.lock
https://github.com/flox/nixpkgs?rev=7fd4004d0511991fff3a0524856a9be09e28c5f0
python3
/nix/store/0kcws5blwbdyxqxjd1w8hjipz61xny3k-python3-3.14.7
$ nix build 'github:NixOS/nixpkgs/7fd4004d0511991fff3a0524856a9be09e28c5f0#python3' --no-link
--print-out-paths
/nix/store/0kcws5blwbdyxqxjd1w8hjipz61xny3k-python3-3.14.7
```

NIX, PART 4

All of it is open source.

Nix	LGPL-2.1	The store, the builder, the sandbox, the language.	github.com/NixOS/nix
nixpkgs	MIT	100,000+ package recipes. The commit your lock points at.	github.com/NixOS/nixpkgs
flox	GPL-2.0	The CLI: manifest, lock, activate, build, containerize.	github.com/flox/flox
flox-skills	Apache-2.0	The agent skills used in the last demo.	github.com/flox/flox-skills

Hosted by Flox: the catalog that resolves names to versions, and FloxHub behind flox push. Your environment is two text files in your repo, and once locked, activate and build need only Nix and a binary cache.

DEMO 1 OF 3

One file.

```
laptop · ~/deterministic-builds-google                    aarch64-darwin

$ flox init
⚡ Created environment 'deterministic-builds-google' (aarch64-darwin)

Next:
  $ flox search <package>    <- Search for a package
  $ flox install <package>   <- Install a package into an environment
  $ flox activate            <- Enter the environment
  $ flox edit                <- Add environment variables and shell hooks
$ flox install python3 redis jq claude-code
✔ 'python3', 'redis', 'jq', 'claude-code' installed to environment 'deterministic-builds-
google'
ℹ 'jq' has additional outputs, use 'flox list -a' to see more
! The package 'claude-code' has an unfree license, please verify the licensing terms of use
$ flox list
claude-code: claude-code (2.1.296)
jq: jq (1.8.2)
python3: python3 (3.14.7)
redis: redis (8.10.2)
```

laptop · ~/deterministic-builds-google aarch64-darwin

```
$ cat .flox/env/manifest.toml
schema-version = "1.18.0"

[install]
python3.pkg-path = "python3"          # runtime
python3.version  = "3.14"
redis.pkg-path   = "redis"            # database
jq.pkg-path      = "jq"
claude-code.pkg-path = "claude-code"  # the agent's harness

[vars]
APP_PORT   = "8080"
REDIS_PORT = "6379"

[services.redis]
command = "redis-server --port $REDIS_PORT --dir $FLOX_ENV_CACHE/redis --save ''"
[services.app]
command = "python3 app.py"

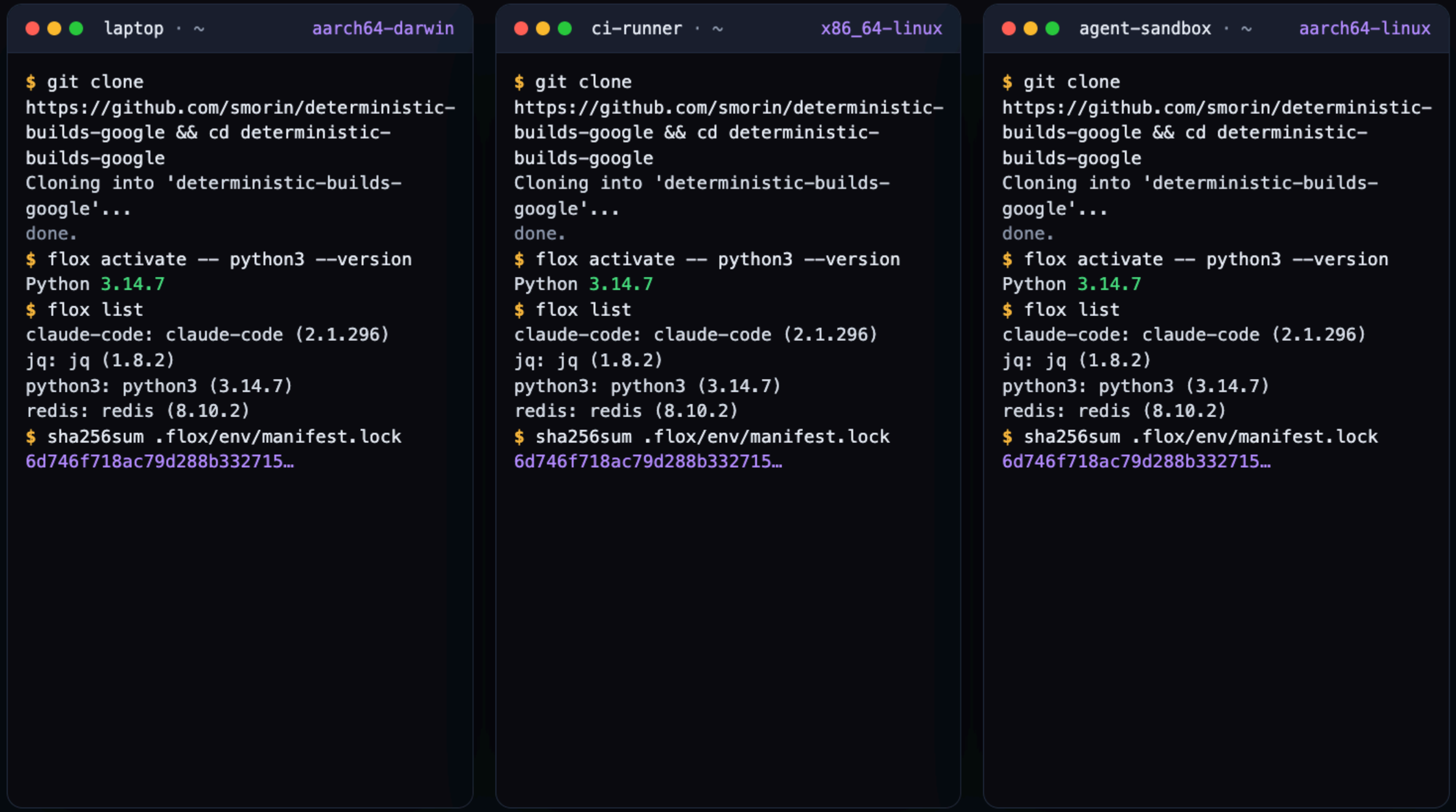
[build.app]
version = "0.1.0"
command = "mkdir -p $out/bin && cp app.py $out/bin/app && chmod +x $out/bin/app"
sandbox = "pure"                      # hermetic
runtime-packages = ["python3", "redis"]

[options]
systems = ["aarch64-darwin", "aarch64-linux", "x86_64-linux"]
```

```
laptop · ~/deterministic-builds-google                    aarch64-darwin
To stop using this environment, run 'flox deactivate'
✔ Service 'app' started.
✔ Service 'redis' started.
[flox] $ python3 --version && redis-server --version && claude --version
Python 3.14.7
Redis server v=8.10.2 sha=00000000:0 malloc=libc bits=64 build=d9f021fa7678041b
2.1.296 (Claude Code)
[flox] $ flox services status
NAME       STATUS       PID
app        Running    97508
redis      Running    97510
[flox] $ curl -s localhost:8080
{
  "service": "ato-demo",
  "version": "0.1.0",
  "python": "3.14.7",
  "system": "arm64-darwin",
  "flox_env": "~/deterministic-builds-google/.flox/run/aarch64-darwin.deterministic-builds-
google-dev",
  "redis": "PONG"
}
```

DEMO 2 OF 3

Three machines, one hash.


laptop · ~
aarch64-darwin
$ git clone https://github.com/smorin/deterministic-builds-google && cd deterministic-builds-google
Cloning into 'deterministic-builds-google'...
done.
$ flox activate -- python3 --version
Python 3.14.7
$ flox list
claude-code: claude-code (2.1.296)
jq: jq (1.8.2)
python3: python3 (3.14.7)
redis: redis (8.10.2)
$ sha256sum .flox/env/manifest.lock
6d746f718ac79d288b332715…
ci-runner · ~
x86_64-linux
$ git clone https://github.com/smorin/deterministic-builds-google && cd deterministic-builds-google
Cloning into 'deterministic-builds-google'...
done.
$ flox activate -- python3 --version
Python 3.14.7
$ flox list
claude-code: claude-code (2.1.296)
jq: jq (1.8.2)
python3: python3 (3.14.7)
redis: redis (8.10.2)
$ sha256sum .flox/env/manifest.lock
6d746f718ac79d288b332715…
agent-sandbox · ~
aarch64-linux
$ git clone https://github.com/smorin/deterministic-builds-google && cd deterministic-builds-google
Cloning into 'deterministic-builds-google'...
done.
$ flox activate -- python3 --version
Python 3.14.7
$ flox list
claude-code: claude-code (2.1.296)
jq: jq (1.8.2)
python3: python3 (3.14.7)
redis: redis (8.10.2)
$ sha256sum .flox/env/manifest.lock
6d746f718ac79d288b332715…

agent-sandbox · /workspace aarch64-linux

```
$ flox activate -r smorin/deterministic-builds-google -- python3 app.py --report
{
  "service": "ato-demo",
  "version": "0.1.0",
  "python": "3.14.7",
  "system": "aarch64-linux",
  "flox_env": "/root/.cache/flox/run/aarch64-linux.deterministic-builds-google-dev",
  "redis": "down"
}
```

DEMO 3 OF 3

The artifact, and the receipt.

```
laptop · ~/deterministic-builds-google                    aarch64-darwin

$ flox build
Building app-0.1.0 in Nix sandbox (pure) mode
app> + mkdir -p /nix/store/cy1f0px21a94qw2wm3vlx5x8iyak3fjz-app-0.1.0/bin
app> + cp app.py /nix/store/cy1f0px21a94qw2wm3vlx5x8iyak3fjz-app-0.1.0/bin/app
app> + chmod +x /nix/store/cy1f0px21a94qw2wm3vlx5x8iyak3fjz-app-0.1.0/bin/app
app> interpreter directive changed from "#!/usr/bin/env python3"
app>   to "/nix/store/sr61jjs5h39akm49m89jfxh923fiv7b5-environment-build-app/bin/python3"
Completed build of app-0.1.0 in Nix sandbox mode

⚡ Built outputs:
./result-app, ./result-app-buildCache, ./result-app-log
$ readlink result-app
/nix/store/cy1f0px21a94qw2wm3vlx5x8iyak3fjz-app-0.1.0
$ ./result-app/bin/app --version
ato-demo 0.1.0 (python 3.14.7)
```

```
laptop · ~/deterministic-builds-google                    aarch64-darwin

$ flox containerize --runtime docker
⚡ 'deterministic-builds-google:latest' written to Docker runtime
$ docker run --rm -v "$PWD:/w" -w /w deterministic-builds-google:latest python3 app.py --
report
{
  "service": "ato-demo",
  "version": "0.1.0",
  "python": "3.14.7",
  "system": "aarch64-linux",
  "flox_env": "/nix/store/103xx4pfsrj5sgm26byam3lcm17vyfwg-environment-dev",
  "redis": "down"
}
$ python3 scripts/sbom.py | jq -r '.packages[] | "\(.name)@\(.versionInfo)  \
(.licenseConcluded)"'
claude-code@2.1.296  Unfree
jq@1.8.2  MIT
python3@3.14.7  Python-2.0
redis@8.10.2  AGPL-3.0-only
```

HUMANS AND AGENTS

One definition, shared.

People

`flox init` once

`flox activate -s` every morning

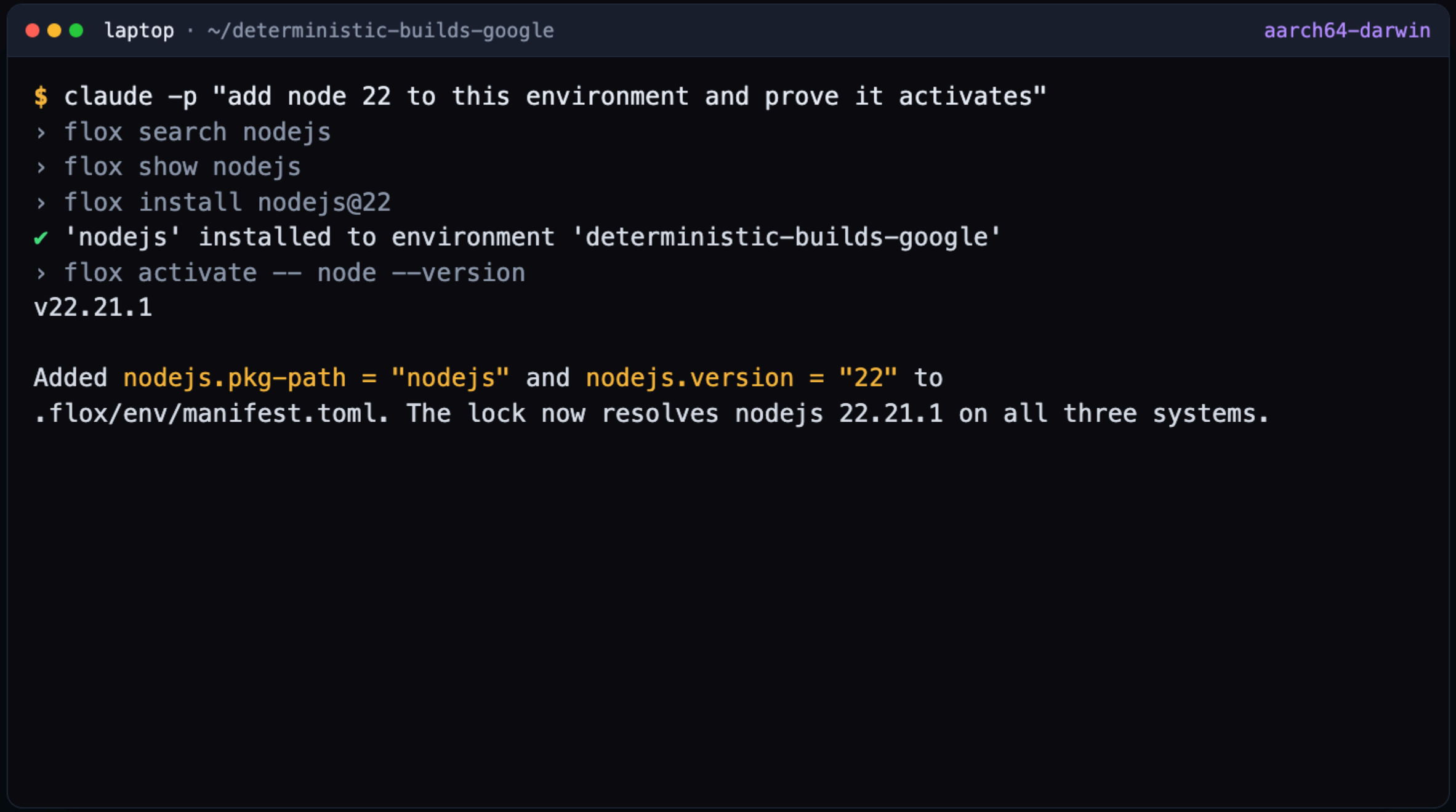
`flox push` to share it

Agents

The `flox` and `floxify` skills write the same manifest

Verify with `flox activate -- <cmd>`

No ad hoc installs: the manifest is the only input


laptop · ~/deterministic-builds-google
aarch64-darwin
$ claude -p "add node 22 to this environment and prove it activates"
› flox search nodejs
› flox show nodejs
› flox install nodejs@22
✔ 'nodejs' installed to environment 'deterministic-builds-google'
› flox activate -- node --version
v22.21.1
Added nodejs.pkg-path = "nodejs" and nodejs.version = "22" to
.flox/env/manifest.toml. The lock now resolves nodejs 22.21.1 on all three systems.

Software is built everywhere now. Make everywhere mean the same thing.

Declare it once. Lock it. Reproduce it bit-for-bit on every machine, for every human and every agent.

Thank you.

- **Nix and nixpkgs**
  nixos.org · github.com/NixOS/nixpkgs
- **Flox CLI (GPL-2.0)**
  github.com/flox/flox · flox.dev
- **Agent skills (Apache-2.0)**
  github.com/flox/flox-skills
- **This demo + deck**
  github.com/smorin/deterministic-builds-google
- **Me**
  @smorin